

Verify existing Docker images:

```
test@ubuntu:~/Desktop$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    a6bd71f48f68  2 weeks ago   187MB
```

Now verify existing Docker containers:

```
test@ubuntu:~/Desktop$ sudo docker ps -a
CONTAINER ID   IMAGE          COMMAND
CREATED        STATUS        PORTS
NAMES
9089d27e1a20   nginx         "/docker-entrypoint..." 34
seconds ago   Up 33 seconds  0.0.0.0:9999->80/tcp,
:::9999->80/tcp  peaceful_torvalds
```

So, what is a Dockerfile?

Dockerfile is the text document that provides build instructions to build the image with your application package.

```
FROM openjdk:11.0-jdk-slim
MAINTAINER osfy
COPY target/my-app-1.0-SNAPSHOT.jar app.jar
ENTRYPOINT ["java","-cp","/app.jar", "com.osfy.app.App"]
```

Let's understand the commands given above.

- **FROM** command: Uses the base image (openjdk:11) for subsequent instructions in Dockerfile.

- **COPY** command: Copies a file to the container.
- **ENTRYPOINT** command: Specifies entry point and manipulates the environment.

Place the Dockerfile in the project root directory. Now it is time to build a Docker image.

Building a Docker image for Java app

Execute the *docker image* command to create an image as shown below:

```
test@ubuntu:~/Desktop/my-app$ sudo docker image build -t my-app:latest
```

```
[+] Building 122.6s (7/7) FINISHED
    docker:default
=> [internal] load build definition from Dockerfile
    0.0s
=> => transferring dockerfile: 183B
    0.0s
=> [internal] load .dockerignore
    0.0s
=> => transferring context: 2B
    0.0s
=> [internal] load metadata for docker.
io/library/openjdk:11.0-jdk-slim
3.2s
=> [internal] load build context
    0.0s
```

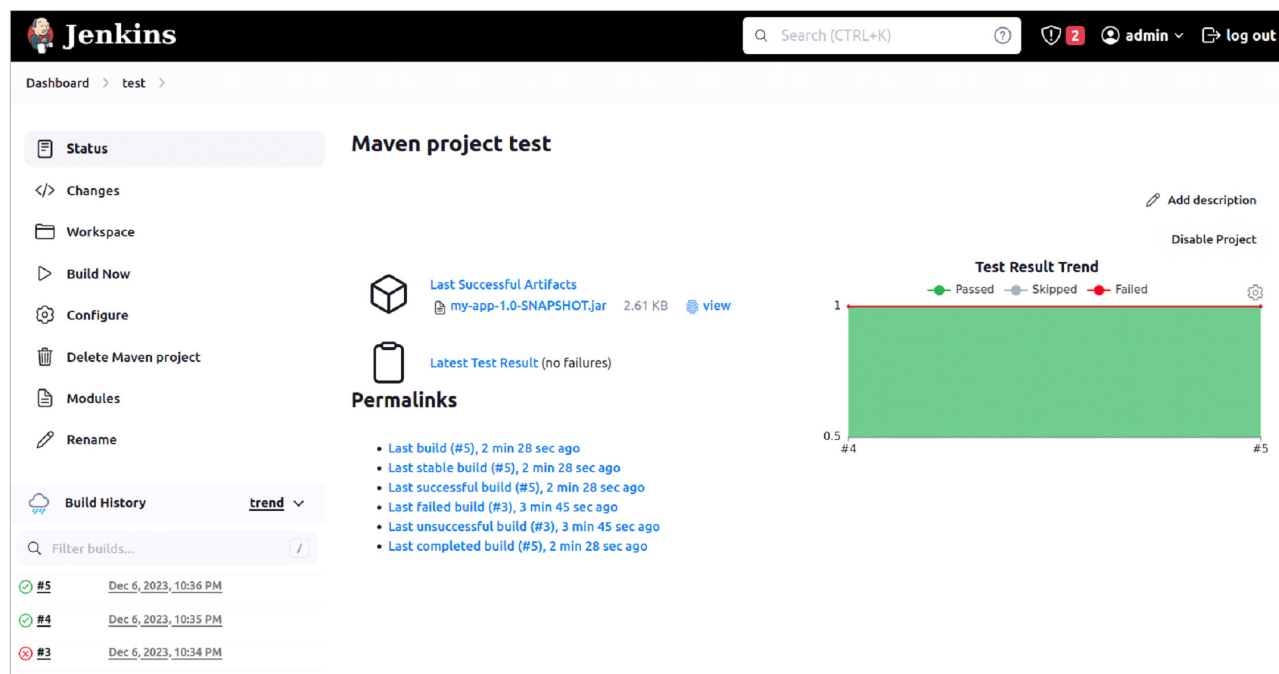


Figure 7: Maven project dashboard